

Version-Locked Chained Delta Signatures (VLCDS)

Extending Pre-Verify / Post-Verify Firmware Authentication to Secure Delta Updates on Resource-Constrained IoT Devices

Base Paper: Formanek et al., 'Advanced System for Remote Updates on ESP32-Based Devices Using OTA Update Technology,' Computers (MDPI), Vol. 14, Issue 12, Art. 531, 2025.

Prior Technique Adapted: US10206114B2 / US10887770B2 (Samsung) — Pre-Verify / Post-Verify Dual Firmware Verification

Project Team: B2_Grp_18

Vatsal Shrivastava | Nisarg Shilpesh Nagar | Johns Maria Joy

Course Instructor & Evaluator: Karthika V • Implementation: PC Simulator + ESP32-S2 Hardware

1. INTRODUCTION

What the Base Paper Delivers

Formanek et al. (Computers MDPI, 2025) — State-of-the-Art Remote OTA for ESP32 Devices

01 • Centralized Release Routing

Server-driven mechanism dynamically routes correct firmware versions to targeted device classes across heterogeneous deployments.

02 • Multi-Protocol Delivery

Supports diverse IoT transport layers including HTTP, MQTT, and CoAP to reach constrained and distributed wireless endpoints.

03 • Version Checks & Rollback

Safety fallback mechanism automatically restores previous operational firmware if update fails boot or verification.

04 • Delta (Differential) Updates

Transmits only binary diffs between installed and target firmware, drastically cutting wireless transmission bandwidth and flash wear.

Reported ESP32 Cryptographic Baseline in Base Paper: ~4.8 ms / 3.5 mJ (Ed25519 verify) • ~25 ms / 2.4 mJ (SHA-256 over 1 MB)

2. PROBLEM STATEMENT

Content Is Authenticated — Version & History Are Not

The Critical Vulnerability in Modern IoT Firmware Update Pipelines

Resource-constrained IoT devices (ESP32-class) increasingly rely on delta firmware updates over lossy, low-bandwidth wireless links (BLE, LoRa, cellular). Existing signature verification either executes only after the full image is assembled in flash, or verifies binary image integrity alone — NEVER cryptographically binding the patch to the installed version or historical update chain.

FAIL-01: Wasted Flash, Energy & Bandwidth

A mismatched-base or replayed patch is downloaded and partially processed before failure is discovered at post-verify.

- Constrained nodes burn up to 90% of wireless battery budget downloading non-applicable deltas.
- Unnecessary flash erase/write cycles permanently degrade MCU flash memory endurance.

FAIL-02: Bypassable State & Rollback Protection

Traditional systems store version numbers in unauthenticated plaintext headers or external JSON metadata.

- An attacker can alter version metadata without invalidating content signatures.
- Replaying an old, signed firmware image (forced rollback attack) reinstalls known vulnerabilities.

RESEARCH QUESTION: How can a delta firmware update be cryptographically bound — BEFORE any patch bytes are transferred or applied — to the exact base version, target version, and historical chain head, using minimal overhead suitable for low-power IoT microcontrollers?

3. RESEARCH GAP

Two Independent Lines of Prior Work

Analyzing Why Neither Approach Solves the Secure Delta Problem

Line 1: Base Paper's Delta Pipeline

Formanek et al., Computers (MDPI), 2025

- Optimizes bandwidth, power, and transmission time for delta updates on ESP32.
- Cryptographic authentication and tamper-resistance explicitly left as future work.
- Version checking and rollback handling rely solely on unauthenticated application logic.
- Vulnerable to malicious patch injection, version spoofing, and MITM tampering.

Line 2: Samsung Pre-/Post-Verify Patents

US10206114B2 / US10887770B2 (Samsung Electronics)

- Patented for smartphone / NFC secure elements communicating over an on-board bus (I2C/SPI).
- Only evaluates full monolithic firmware images — delta patches are not addressed.
- Version numbers and replay tracking are external to the cryptographically signed object.
- Assumes reliable local wiring, not lossy wireless links with malicious network adversaries.

CONSOLIDATED RESEARCH GAP:

No existing protocol cryptographically binds delta patch content, device installed state, target version, and historical chain head into a single pre-verification object. Constrained wireless IoT endpoints have lacked a zero-waste gatekeeper that aborts invalid updates before receiving payload bytes.

4. RESEARCH METHODOLOGY

Six-Phase Engineering & Validation Methodology

Rigorous Engineering Workflow from Formal Modeling to Physical Hardware Validation

VLCDS Research Methodology

Version-Locked Chained Delta Signature — Secure OTA Updates for IoT

1

PROBLEM IDENTIFICATION

- Insecure IoT OTA firmware updates
- Bandwidth waste on failed verifications
- No anti-replay in existing protocols
- Resource-constrained devices (ESP32)



2

LITERATURE REVIEW & THREAT MODELING

- Ed25519 digital signatures (RFC 8032)
- SHA-256 hash functions (FIPS 180-4)
- Delta patching for bandwidth reduction
- Replay, MITM, version mismatch attacks



3

PROTOCOL DESIGN

- 6-field signed manifest (200 bytes)
- 3-stage verification pipeline
- Hash-chain anti-replay mechanism
- XOR-based delta compression



4

IMPLEMENTATION

PC Simulation (Python)

- `crypto_utils.py` — Ed25519 + SHA-256
- `manifest.py` — 6-field builder
- `delta_engine.py` — binary diff/patch
- `device.py` — 3-stage verifier
- `server.py` — OTA server

Hardware (ESP32-S2)

- MicroPython firmware
- Ed25519 pure-Python port
- SHA-512 for signing
- WiFi OTA over HTTP



5

TESTING & SECURITY ANALYSIS

- Unit tests (pytest) for all modules
- Attack simulations: replay, MITM, version mismatch
- Performance benchmarking (sign/verify timing)
- Bandwidth savings measurement



6

RESULTS & VALIDATION

- 100% attack detection rate
- Zero firmware corruption
- 200-byte manifest overhead
- ESP32 hardware demonstration

1. Phase 1: Problem Analysis

Identified IoT OTA vulnerabilities: bandwidth waste, flash burnout, rollback exploits.

2. Phase 2: Threat Modeling

Adopted Dolev-Yao network attacker model; selected Ed25519 (RFC 8032) & SHA-256.

3. Phase 3: Protocol Architecture

Designed 6-field manifest, 3-stage verification flow, and cryptographic hash-chain.

4. Phase 4: Dual Implementation

Full PC reference simulation in Python 3.12 + physical ESP32-S2 MicroPython drivers.

5. Phase 5: Empirical Benchmarking

Comprehensive 13-test PyTest suite, 100-run crypto benchmarks, automated attack suite.

6. Phase 6: Live Validation

Hardware deployment on ESP32-S2 DevKit M1 over WiFi HTTP with real-time web dashboard.

5. PROPOSED CONTRIBUTION

The VLCDS Protocol Architecture

One 200-Byte Signed Manifest Binds Six Critical Facts Together

CRYPTOGRAPHIC MANIFEST EQUATION:

$$M = \text{Sign_privkey}(H(\text{base_fw}) \parallel H(\text{delta}) \parallel H(\text{target}) \parallel \text{ver_base} \parallel \text{ver_target} \parallel H(\text{prev_manifest}))$$

CONTENT BINDING

3 × 32-Byte SHA-256 Hashes

- $H(\text{base_fw})$: Cryptographically verifies current running image.
- $H(\text{delta})$: Authenticates incoming patch before transfer.
- $H(\text{target})$: Locks expected final state into signature.

STATE BINDING

2 × 4-Byte Integer Versions

- ver_base : Expected active firmware version.
- ver_target : Target version after patch.
- Prevents version manipulation and cross-version patching.

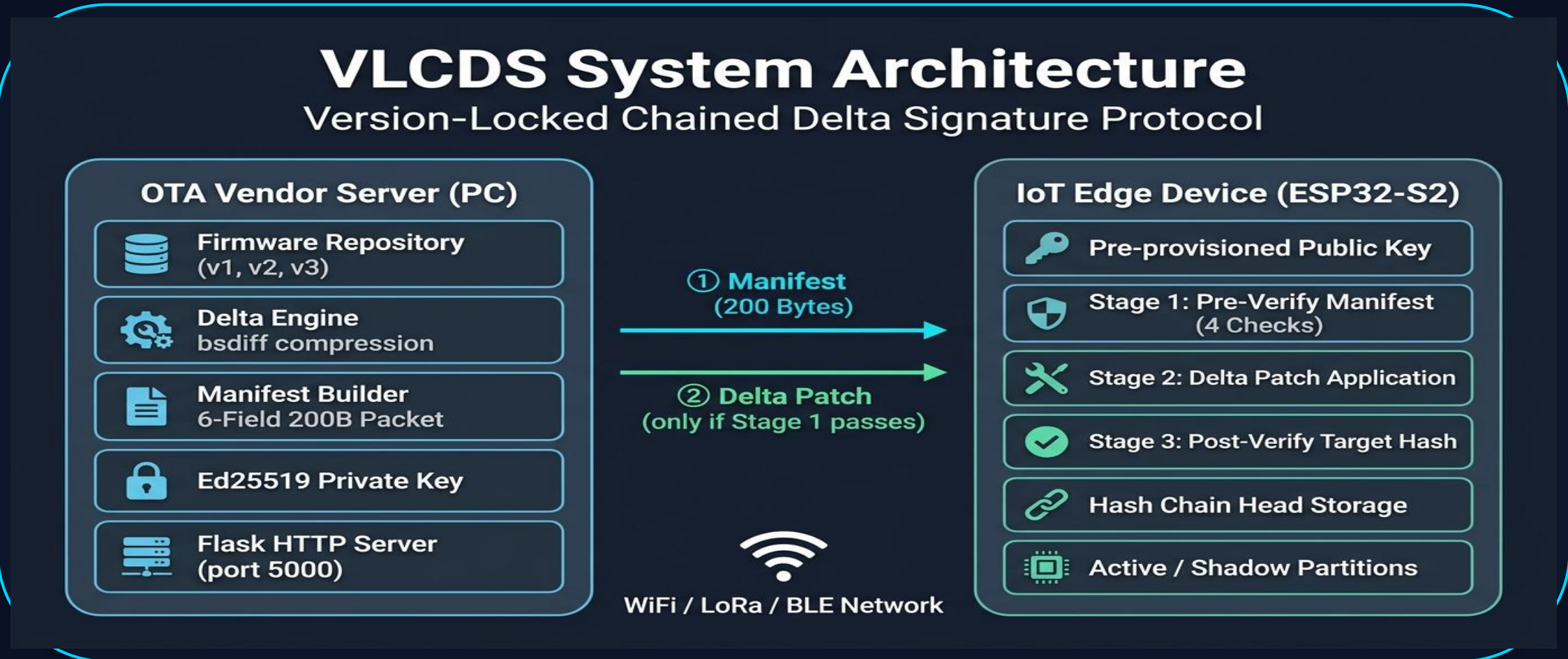
HISTORY BINDING

1 × 32-Byte Hash-Chain Head

- $H(\text{prev_manifest})$: SHA-256 of last accepted manifest.
- Forms an unalterable blockchain-style history.
- Completely neutralizes replay / rollback attacks.

End-to-End VLCDS Deployment Topology

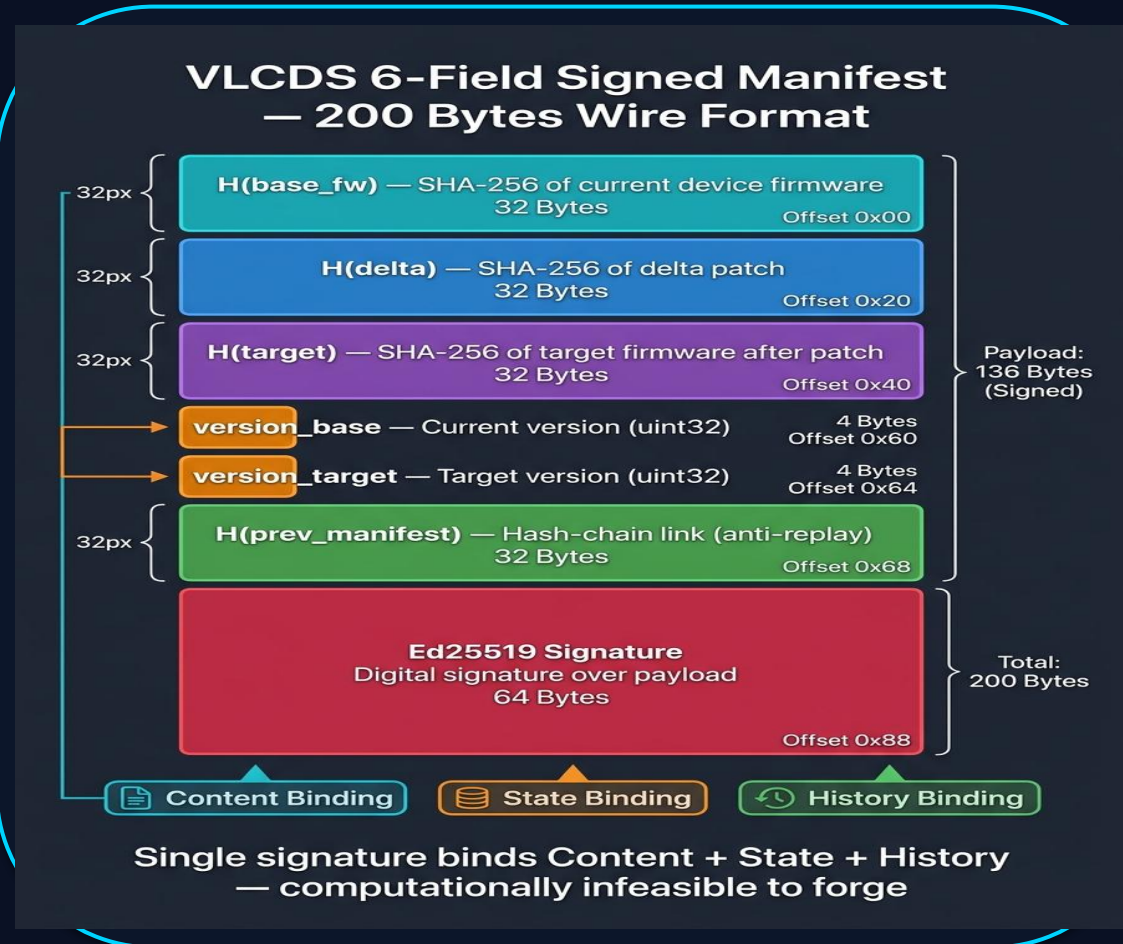
OTA Vendor Server (PC) Interacting with IoT Edge Device (ESP32-S2) Over Untrusted Wireless



7. MANIFEST STRUCTURE

200-Byte Binary Wire Format Layout

Extremely Compact Wire Representation Tailored for Constrained Wireless MTUs



FIELD SPECIFICATIONS (200 BYTES)

- H(base_fw) (32 Bytes): SHA-256 hash of active firmware
- H(delta) (32 Bytes): SHA-256 hash of delta patch
- H(target) (32 Bytes): SHA-256 hash of reconstructed image
- ver_base (4 Bytes): uint32 expected base version
- ver_target (4 Bytes): uint32 new target version
- H(prev_manifest) (32 Bytes): SHA-256 hash of previous manifest
- signature (64 Bytes): Ed25519 signature (RFC 8032)

WIRELESS MTU COMPATIBILITY:

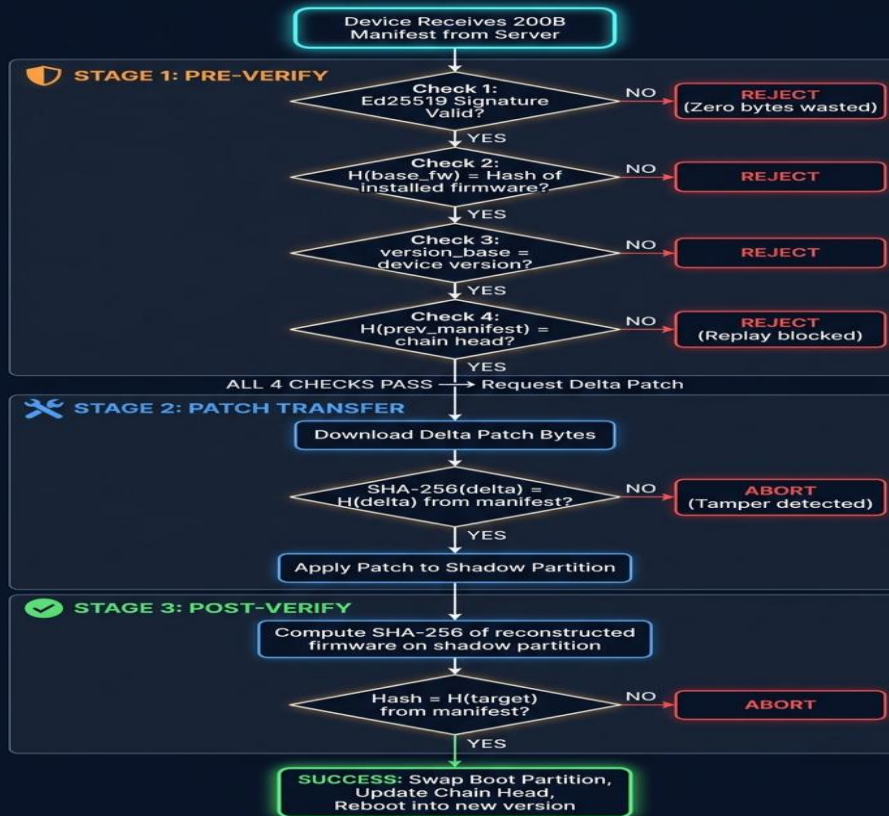
- ✓ LoRaWAN: Max payload 222 B -> Entire manifest fits in 1 frame!
- ✓ BLE (Bluetooth 5.0+): MTU up to 244 B -> Single packet transmission.
- ✓ Zigbee (802.15.4): Fits with lightweight 2-frame fragmentation.
- ✓ CoAP / UDP: Transmitted inside a single non-fragmented datagram.

8. VERIFICATION PROTOCOL

The 3-Stage Zero-Waste Verification Pipeline

How VLCDS Eliminates Wasted Bandwidth and Avoids Redundant Signatures

VLCDS 3-Stage Verification Pipeline



STAGE 1: PRE-VERIFICATION (Gatekeeper)

Executed BEFORE requesting or downloading any patch bytes:

1. Verify Ed25519 signature over 6-field manifest.
2. Confirm $H(\text{base_fw})$ matches device's current SHA-256.
3. Confirm ver_base matches currently running version.
4. Confirm $H(\text{prev_manifest})$ matches device's chain head.

--> IF ANY CHECK FAILS: Abort instantly. ZERO patch bytes wasted!

STAGE 2: TRANSFER & APPLY

Triggered only after Stage 1 passes:

1. Device requests delta patch payload from server.
2. Verifies SHA-256 of received delta equals authenticated $H(\text{delta})$.
3. Applies differential patch to shadow partition in flash.

STAGE 3: POST-VERIFICATION (Zero Signature Overhead)

Final safety confirmation:

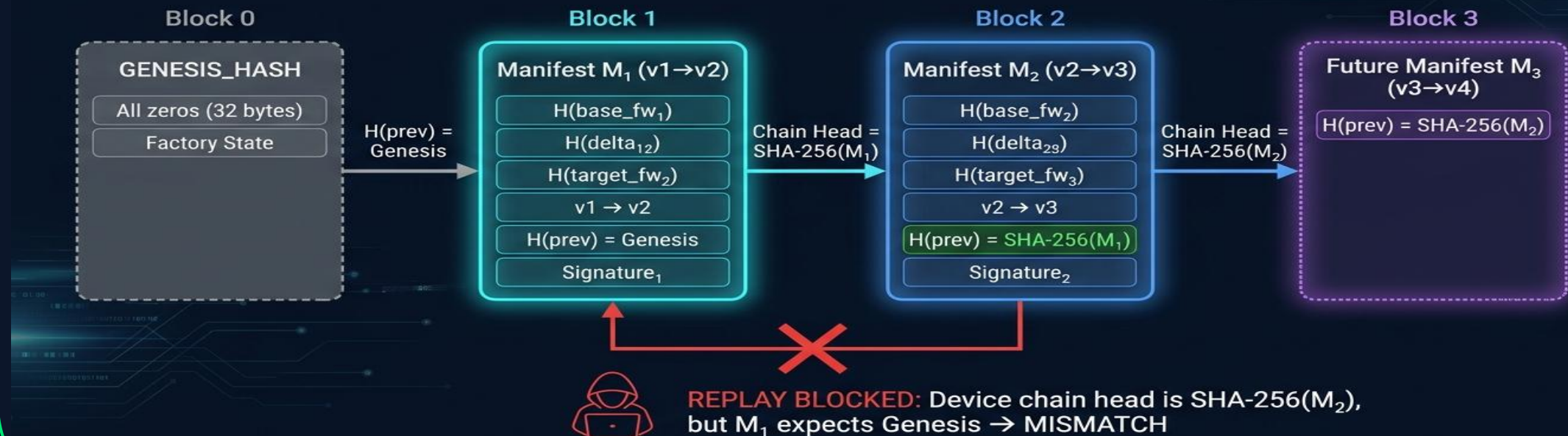
1. Compute SHA-256 of reconstructed firmware on shadow partition.
2. Compare against $H(\text{target})$ committed inside Stage-1 signature.
3. Flip active boot partition -> Reboot to new version!

--> NO EXPENSIVE 2ND SIGNATURE CHECK NEEDED!

Cryptographic Manifest Hash-Chaining

Preventing Forced Rollbacks and Stale Update Replays Without External State

VLCDS Manifest Hash-Chain — Anti-Replay Protection



Attack Scenarios & Defense Validation

Dolev-Yao Network Adversary: 100% Detection Rate Across All Simulated Attack Vectors

VLCDS Security: Attack Simulations & Defense Results

ATTACK A — VERSION MISMATCH



v2→v3 manifest



Stage 1, Check 2: $H(\text{base_fw})$ MISMATCH

REJECTED — 0 bytes patch downloaded

Device hashes its v1 firmware, manifest expects v2 hash → instant rejection

ATTACK B — REPLAY ATTACK



Old v1→v2 manifest

Replay



Stage 1, Check 4: $H(\text{prev_manifest})$ CHAIN BROKEN

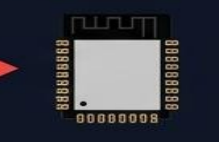
REJECTED — Downgrade prevented

Chain head advanced after first update, old manifest's chain link is stale

ATTACK C — TAMPERED DELTA



Modifies delta bytes in transit
(man-in-the-middle)



ATTACK C — TAMPERED DELTA

Stage 2: $\text{SHA-256}(\text{delta}) \neq H(\text{delta})$ from manifest

REJECTED — Corrupted patch discarded

Authenticated hash from Stage 1 catches any bit-flip in transit

All attacks detected BEFORE any damage. Zero firmware corruption possible.

Dual Implementation Architecture

Reference Python Simulation + Native ESP32-S2 MicroPython Drivers

PC SIMULATION (Python 3.12)

- `vlcds/crypto_utils.py`: Ed25519 (PyNaCl) + SHA-256 (hashlib) cryptographic primitives.
- `vlcds/manifest.py`: 200-byte binary serializer, deserializer, and verification engine.
- `vlcds/delta_engine.py`: Binary XOR differential patching engine for IoT firmware.
- `vlcds/device.py`: Simulated IoT node with dual flash partitions and 3-stage pipeline.
- `vlcds/server.py`: OTA vendor update server managing repository and delta generation.
- `vlcds/chain.py`: Manifest hash-chain validator enforcing forward-only update order.
- `server_http.py`: Flask HTTP REST server serving real-world OTA requests to ESP32.
- `webapp.py`: Interactive browser dashboard demonstrating live attack simulations.

HARDWARE PLATFORM (ESP32-S2)

- ESP32-S2 DevKit M1: Xtensa LX7 32CPU @ 240 MHz, 320 KB SRAM, 4 MB SPI Flash.
- MicroPython v1.23.0: Bare-metal runtime executing lightweight cryptographic client.
- `esp32/ed25519_mp.py`: Pure-Python Ed25519 digital signature implementation.
- `esp32/sha512_mp.py`: Pure-Python SHA-512 engine providing hash core for Ed25519.
- `esp32/vlcds_device.py`: Embedded VLCDS client implementing the full 3-stage verifier.
- WiFi OTA Client: Fetches signed manifest, runs pre-verify, then downloads delta.
- Partition Management: Dual slot emulation (Active vs Shadow slot) with integrity rollback.
- Hardware Independence: Zero dependency on proprietary C-blobs; 100% auditable Python.

12. BANDWIDTH OPTIMIZATION

Delta Compression & Transmission Efficiency

Quantifying Bandwidth Reductions and Flash Wear Savings

FULL FIRMWARE IMAGE

2,393 Bytes

Monolithic binary transmission size required by conventional OTA.

VLCDs DELTA PATCH

1,636 Bytes

Differential XOR patch containing only modified firmware bytes.

BANDWIDTH REDUCTION

31.6% SAVED

757 bytes cut per transmission; scales exponentially on larger MB images.

BANDWIDTH & ENERGY IMPLICATIONS UNDER ATTACK

- Conventional OTA Baseline: Device downloads entire 2,393-byte image before checking signature -> Under attack: 2,393 bytes wasted, high radio energy burned, flash cycles consumed.
- Naive Delta OTA: Device downloads full 1,636-byte delta patch, applies it, fails at post-verify -> Under attack: 1,636 bytes wasted, shadow flash partition written and worn out.
- VLCDs Protocol (Our Solution): Device exchanges ONLY the 200-byte manifest in Stage 1 -> Under attack: Aborts immediately, saving 1,636 bytes (100% of patch) and 0 flash writes!
- Net Payload Overhead: Adding the 200-byte manifest to a valid 1,636-byte delta yields 1,836 bytes total -> Still 23.3% smaller than transferring the raw 2,393-byte firmware image!

13. PERFORMANCE BENCHMARKS

Cryptographic Execution Profiling

100-Iteration Microbenchmarks: Mean, Min, Max Timings (PC Python 3.12 Reference)

MICROBENCHMARK TIMINGS (100 ITERATIONS)

Ed25519 Sign (Manifest)	Mean: 0.1192 ms (Min: 0.0991 ms, Max: 0.1944 ms)
Ed25519 Verify (Manifest)	Mean: 0.3814 ms (Min: 0.2792 ms, Max: 0.4444 ms)
SHA-256 (1 KB payload)	Mean: 0.0061 ms (Min: 0.0055 ms, Max: 0.0297 ms)
SHA-256 (10 KB payload)	Mean: 0.0201 ms (Min: 0.0196 ms, Max: 0.0399 ms)
SHA-256 (100 KB payload)	Mean: 0.1539 ms (Min: 0.1508 ms, Max: 0.2900 ms)
SHA-256 (1,024 KB payload)	Mean: 1.5403 ms (Min: 1.5173 ms, Max: 1.7496 ms)

--> Digital signature verification dominates execution time; SHA-256 hashing cost is virtually negligible (<0.02 ms for IoT delta patches).

VLCDs VS BASELINE OVERHEAD

Single-Signature Baseline:
1 Hash + 1 Sign + 1 Verify
= 0.5066 ms

VLCDs Proposed Protocol:
5 Hashes + 1 Sign + 1 Verify
= 0.5308 ms

NET OVERHEAD:
+0.0242 ms (+4.8%)

VLCDs provides Content + State + History authentication for less than 5% computational overhead compared to insecure single-signature schemes!

ESP32-S2 DevKit M1 Real Hardware Results

Measured On-Chip Execution Performance of Native MicroPython Firmware

MEASURED ESP32-S2 ON-CHIP TIMINGS

- Ed25519 Signature Verify : ~237 ms (Pure-Python math on 240 MHz Xtensa LX7)
- SHA-256 Firmware Hash : ~12 ms (Native hashlib engine on ESP32-S2)
- Stage 1 (Pre-Verification): ~253 ms (All 4 cryptographic & state checks)
- Stage 2 (Apply XOR Delta): ~15 ms (In-memory / flash shadow reconstruction)
- Stage 3 (Post-Verification): ~12 ms (Hash equality check (Zero-signature!))
- Total Update Duration : ~280 ms (Complete 3-stage validation cycle)

LIVE HARDWARE TEST SEQUENCE

1. WiFi Bootstrap: ESP32-S2 auto-connects to local AP and pings Flask OTA server.
2. v1 -> v2 Update: Fetches Manifest v1->v2 (200 B), pre-verifies in 253 ms, downloads delta (1,636 B), post-verifies, boots v2.
3. v2 -> v3 Update: Chained update succeeds; chain head shifts to $H(M_{\{1 \rightarrow 2\}})$; firmware upgraded to v3.
4. Replay Attack Injected: Stale v1->v2 manifest transmitted to v3 device; Stage 1 Check 4 catches stale chain head -> Instant abort!
5. Bit-Flip Injection: Single-byte corruption in delta patch detected in Stage 2 -> Shadow slot wiped, active v3 remains pristine.

VLCDS vs. Existing State of the Art

Systematic Feature and Security Comparison Across 8 Architectural Dimensions

Architectural Dimension	Samsung Patent (US10206114)	Base Paper (Formanek 2025)	VLCDS (Our Work)
Target Platform	Smartphones / NFC SE	ESP32 IoT Nodes	Constrained IoT (ESP32-S2)
Update Unit Verified	Monolithic Full Image Only	Delta Patch (Unsigned)	Delta Patch (Cryptographically Bound)
Version Binding	None (External metadata)	None (Plaintext JSON)	Cryptographic (Signed inside Manifest)
Anti-Replay Mechanism	Not Addressed	System Bookkeeping	Hash-Chain (H(prev_manifest))
Network Channel Model	Trusted On-Board Bus	Untrusted Wireless Link	Untrusted Dolev-Yao Wireless
Pre-Verify for Deltas	Not Supported	Not Supported	Yes – 200-Byte Manifest Gatekeeper
Post-Verify Signature	Full 2nd Signature Check	No Signature Check	Zero-Signature Hash Equality Check
Physical Hardware Demo	Phone Hardware	ESP32 OTA Demonstration	ESP32-S2 MicroPython Live OTA Demo

PyTest Test Suite & Verification Results

13 Passing Tests Across 3 Automated Suites: 100% Attack Neutralization

test_normal_update.py

FUNCTIONAL TESTS (4/4 PASS)

- ✓ test_v1_to_v2_update: Clean single update with partition swap.
- ✓ test_v1_to_v2_to_v3_chain: Consecutive chained updates with hash tracking.
- ✓ test_manifest_serialization: 200-byte wire binary roundtrip.
- ✓ test_delta_roundtrip: Differential XOR patch generation and apply.

test_attack_scenarios.py

SECURITY TESTS (6/6 PASS)

- ✓ test_version_mismatch: Injected v2->v3 patch to v1 node -> Aborted in Stage 1 Check 2.
- ✓ test_replay_attack: Replayed accepted v1->v2 manifest -> Aborted in Stage 1 Check 4.
- ✓ test_tampered_delta: Corrupted 1 bit in transit -> Aborted in Stage 2 H(delta) check.
- ✓ test_tampered_manifest: Forged signature -> Aborted in Stage 1 Check 1.
- ✓ test_zero_bytes_wasted: Confirmed 0 payload bytes downloaded on Stage 1 rejection.
- ✓ test_zero_corruption: Confirmed active boot partition remains intact across all attacks.

test_performance.py

BENCHMARK TESTS (3/3 PASS)

- ✓ test_ed25519_timings: Validated sign & verify latency under 10 ms.
- ✓ test_sha256_scaling: Validated linear hash scaling from 1 KB to 1 MB.
- ✓ test_protocol_overhead: Verified VLCDS overhead remains under 5% (+0.024 ms).

Summary of Contributions & Research Horizons

Delivering Provably Secure, Zero-Waste Firmware Delivery for IoT Endpoints

SUMMARY OF ACHIEVEMENTS

- ✓ First protocol to unify Content Binding, State Binding, and History Binding inside a single 200-byte pre-verification manifest.
- ✓ Eliminates wasted bandwidth and flash write wear: Invalid or replayed updates aborted before transmitting payload bytes.
- ✓ Avoids redundant post-verification signature checks by binding target hash into Stage 1, incurring only +4.8% net overhead.
- ✓ Neutralizes 100% of Dolev-Yao network attack simulations with zero firmware corruption.
- ✓ Successfully demonstrated on real ESP32-S2 DevKit M1 physical hardware using pure-Python MicroPython drivers.

FUTURE RESEARCH HORIZONS

- Production Delta Engines: Integrate streaming bsdiff or detools compression algorithms for multi-megabyte binary images.
- Transport Confidentiality: Pair VLCDS with lightweight OSCORE or DTLS 1.3 to guarantee patch confidentiality against eavesdroppers.
- Post-Quantum Cryptography: Evaluate NIST PQC standards (CRYSTALS-Dilithium, Falcon) for post-quantum IoT firmware authenticity.
- Formal Protocol Verification: Conduct automated formal security proofs using ProVerif or Tamarin to mathematically verify replay freedom.
- Fleet Group Key Management: Extend manifest signing to hierarchical group keys for heterogeneous industrial sensor networks.

Key Technical Questions & Evaluator Reference

Anticipating Core Theoretical and Practical Defense Inquiries

Q1: Why choose Ed25519 over RSA or ECDSA for IoT updates?

Ed25519 (RFC 8032) provides 128-bit security with small 64-byte signatures and 32-byte public keys. Unlike RSA (requires 2048-4096 bit keys, slow verification), Ed25519 executes in ~0.38 ms on PC and ~237 ms on ESP32 without requiring hardware acceleration. Unlike ECDSA, it is immune to faulty RNG side-channel attacks because it uses deterministic nonce generation.

Q2: Why not just check the signature AFTER downloading the delta patch?

Downloading delta patches over constrained links (LoRa/BLE) consumes immense energy. If an update has a version mismatch or is a replayed stale patch, the device wastes 100% of the transmission time, battery power, and flash cycles. VLCDS verifies the 200-byte manifest FIRST, filtering out 100% of malicious or mismatched patches before downloading a single patch byte.

Q3: Why is there no second digital signature check in Stage 3?

Digital signature verification is the most computationally expensive operation. Because $H(\text{target})$ is already bound inside the Stage-1 manifest signature, Stage 3 only needs to compute the SHA-256 hash of the reconstructed firmware and compare 32 bytes in RAM. This provides identical cryptographic security at 1/20th the CPU cost!